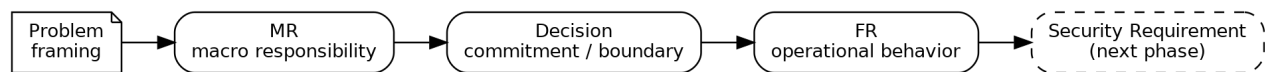


DDTA Authoring Rules Through Functional Requirement

THESIS WRITING CONTRACT - REVISION 2

This document turns the current conceptual baseline into repeatable authoring questions and review invariants. It intentionally stops before formal Base Analysis/BAE and Security Requirement semantics.

1. Authoring sequence



Problem framing



Macro Requirement



Decision



Functional Requirement



Specialized Requirement (next: Security)

The hierarchy is deliberately regular for teachability, validation and tool support.

2. Step 0 - Frame the project problem

Write one short project-level statement before MR decomposition.

Question:

Which general problem or class of problems must the project address if current solution choices are removed?

Rules:

- describe problem and boundary, not the chosen architecture;
- remove current provider/device/database/framework/algorithm names and test whether the statement still makes sense;
- state important out-of-bound responsibilities;
- use the framing to test MR completeness/overlap;
- do not copy the framing into every MR.

3. Macro Requirement authoring

Question:

Within the framed problem, which stable macro responsibility are we governing, what macro result/value must it contribute, why does it matter and who is involved?

Required body semantics:

- **Title** - concise domain-understandable responsibility;
- **Intent** - normally 1-2 sentences on purpose/value/outcome;

- **Context** - minimum interpretive background;
- **Stakeholders** - materially affected/benefiting/governing roles;
- **Scope** - semantic responsibility boundary;
- **Assumptions/Constraints** - optional, only if they apply to the whole branch;
- **dependsOn MR** - when another MR is genuinely complementary/dependent.

Review tests:

- problem anchoring;
- single concern;
- architecture resilience;
- temporal stability;
- solution-removal;
- stakeholder readability;
- no atomic obligation;
- dependency-vs-containment;
- teachability/project-map usefulness.

4. Decision authoring

Question:

Which significant commitment narrows this MR, why must the project take that position, and which consequences do we accept?

Required body semantics:

- **Title**;
- **Context** - unresolved local tension or reason that only one admissible position remains;
- **Decision** - one coherent commitment;
- **Consequences** - benefits, costs, risks, constraints and downstream obligations.

4.1 Responsibility-boundary question

Before FR authoring, explicitly ask:

Does the project implement/own this responsibility internally, or does it consume an external service/provider/organizational capability?

This is usually a genuine Decision because it changes which FRs are project responsibilities.

Example:

MR Warehouse management

```
|
+-- Decision: project-owned stock authority
|   -> internal receipt/adjustment/reservation/issue FRs
|
|-- alternative Decision: external WMS authority
|   -> API/contract/correlation/mapping/retry FRs
|   -> WMS internal concurrency/accounting is outside project FR ownership
```

4.2 Necessity/default Decision - rare normalization rule

If no materially distinct admissible alternative is found, **do not skip the Decision layer**. Write an explicit necessity/default Decision only after review.

Checklist:

1. Context explains why alternatives are absent, immaterial or excluded.

2. Decision states the project commitment, not a tautology.
3. Consequences identify downstream responsibilities.
4. The document is reopenable if constraints/alternatives change.
5. "We did not look for alternatives" is not a valid justification.

This is a deliberate methodological trade-off: a rare normalization node is preferred over a variable hierarchy because it keeps authoring, IDs, validation and tool navigation simpler.

5. Functional Requirement authoring

Question:

Given the parent Decision and an applicable condition/input, which subject/capability must perform which functional action on/with which object/information, and which observable result or failure behavior must follow?

Required semantics:

- **Title** - one coherent operational capability/behavior;
- **Functional obligation** - readable normative prose;
- **Functional clauses** - one or more concrete normative statements;
- **SPO references** - structured subject/predicate/object references supporting the clauses;
- **Parent Decision** - exactly one semantic parent.

FR review tests:

- **operationality** - behavior is concrete, not merely a desirable state;
- **independent assessability** - reviewers can determine whether it holds;
- **parent coverage** - the parent Decision actually establishes the responsibility/strategy this FR operationalizes;
- **responsibility boundary** - switching internal/external ownership would materially change the FR branch;
- **decision non-repetition** - FR operationalizes rather than restates;
- **coherent unit** - independently changeable capabilities are split;
- **implementation independence** - different valid implementations can satisfy the same FR;
- **parameter classification** - mutable values are not hard-coded without semantic justification;
- **reference reuse** - governed entities are referenced, not duplicated;
- **method neutrality** - no threat-method vocabulary is needed to author the FR.

6. SPO usage

SPO is a structured relation aid, not the whole FR.

Example:

Normative clause:

When a full ReservationRequest is allocatable, InventoryService MUST create one ACTIVE Reservation for the originating OrderEvaluation; competing requests MUST NOT reserve the same already-frozen quantity.

SPO references:

```
InventoryService -- creates --> Reservation
InventoryService -- updates --> InventoryBalance
InventoryService -- produces --> ReservationResult
```

The normative clause owns conditions, correlation, atomicity, lifecycle and failure semantics. SPO makes identity/action/object references explicit.

A future authoring tool may:

- suggest existing governed subject/object entries;
- filter FunctionalPredicate choices by compatible kinds;
- detect duplicate references and prose/SPO mismatch;
- start an explicit proposal workflow for a missing entry;
- never silently create canonical project semantics from nouns in prose.

7. FunctionalPredicate vocabulary

The core defines a predicate metacontract, not an exhaustive verb list.

Candidate areas/examples:

Area	Examples
storage	reads, writes, records, retrieves
communication	sends, receives, requests, returns
lifecycle	creates, activates, suspends, releases, expires, purges
transformation	produces, derives, evaluates, maps
inventory	reserves, releases, increases, decreases, blocks
cryptography	encrypts, decrypts, hashes, derivesKey
signature	signs, verifiesSignature, validatesCertificate
identity/access	verifies, authorizes, denies, commands

Every predicate needs stable identity, readable meaning, functional area and compatible subject/object kinds. Formal compatibility waits for the Base Analysis taxonomy.

8. Cross-MR service composition

Two MRs can jointly provide an end-to-end service without sharing FR ownership.

Rules:

- each FR remains inside exactly one MR -> Decision branch;
- a consumer FR may require a service/capability owned by another MR;
- consumption does not create a second parent or owner;
- the exact canonical relation target (provider FR vs service/capability/BAE/interface) is deferred to Base Analysis.

9. No Decisions under FR

If a new design choice appears while implementing/reviewing an FR:

1. create/revise a Decision under the relevant MR;
2. create, revise or supersede the affected FR branch;
3. do not create a Decision as a child of FR.

This keeps the reading model stable:

MR -> Decision -> FR -> SpecializedRequirement

10. Specialized Requirements and thesis scope

Specialized Requirements descend from FR and add domain-specific constraints without replacing the underlying function.

For the thesis end-to-end implementation/evaluation cycle, the next specialization to formalize is:

SecurityRequirement

Expected future chain:

FR

- > Base Analysis
- > overlay/plugin (first concrete test: STRIDE)
- > Analysis Record / Finding
- > accepted Finding
- > SecurityRequirement
- > effective obligation = FR + applicable SecurityRequirements
- > revised design
- > re-analysis

Governance, Privacy and Performance/Quality specializations remain future/generalization work unless a concrete counterexample forces earlier treatment.

11. Parameter rule

Do not introduce a generic functionalParameters field.

Classify values by semantics and reference the appropriate governed artifact/constraint:

- acquisition medium/strategy -> Decision;
- latency/throughput target -> Performance/Quality candidate;
- retention -> Governance/Privacy policy candidate;
- provider API limit -> InterfaceContract candidate;
- tuning/buffer/thread count -> realization/configuration.

12. Reopen rule

The MR/Decision/FR authoring baseline may be reopened if the SecurityRequirement or Base Analysis/overlay phase produces a concrete counterexample that cannot be represented without breaking these invariants.